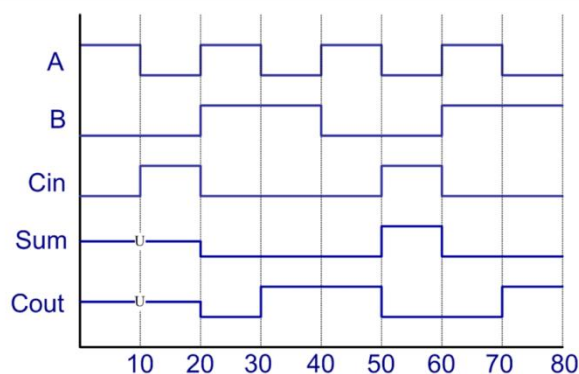


请编写testbench文件，用下图中的A、B、Cin来进行激励。请分别进行behavior simulation、post synthesis simulation、以及post implementation simulation，请根据以下问题分析这些结果的特点和异同。



实验目的:

在 Vivado 中对全加器进行设计仿真，编写 testbench 测试模块功能，分别进行 behavior simulation, post synthesis simulation 和 post implementation simulation，对仿真结果进行分析和比较。

VHDL 代码:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity full_adder is
    Port ( A : in STD_LOGIC;
          B : in STD_LOGIC;
          Cin : in STD_LOGIC;
          Sum : out STD_LOGIC;
          Cout : out STD_LOGIC);
end full_adder;

architecture dataflow of full_adder is
    signal s1, s2, s3 : std_logic;
    constant gate_delay : time := 10 ns;
begin
    L1: s1 <= (A xor B) after gate_delay;
    L2: s2 <= (Cin and s1) after gate_delay;
    L3: s3 <= (A and B) after gate_delay;
    L4: sum <= (s1 xor Cin) after gate_delay;
    L5: cout <= (s2 or s3) after gate_delay;
end dataflow;
```

testbench 代码:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```

entity full_adder_tb is
-- Port ( );
end full_adder_tb;

-- 结构体
architecture tb of full_adder_tb is
--元件声明
component full_adder is
    Port ( A : in STD_LOGIC;
           B : in STD_LOGIC;
           Cin : in STD_LOGIC;
           Sum : out STD_LOGIC;
           Cout : out STD_LOGIC);
end component;
--信号声明
signal a_i, b_i, c_i, s_o, c_o : std_logic;
constant period : time := 10ns;

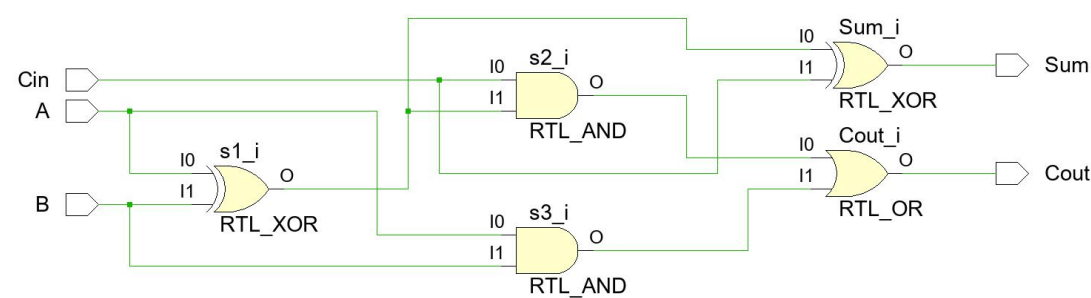
begin
--实例化
uut: full_adder port
    map( A => a_i,
         B => b_i,
         Cin => c_i,
         Sum => s_o,
         Cout => c_o);
--测试用例
a_i <= '1', '0' after period, '1' after period*2,
        '0' after period*3, '1' after period*4,
        '0' after period*5, '1' after period*6,
        '0' after period*7, '1' after period*8;
b_i <= '0', '1' after period*2, '0' after period*4,
        '1' after period*6;
c_i <= '0', '1' after period, '0' after period*2,
        '1' after period*5, '0' after period*6;

end tb;

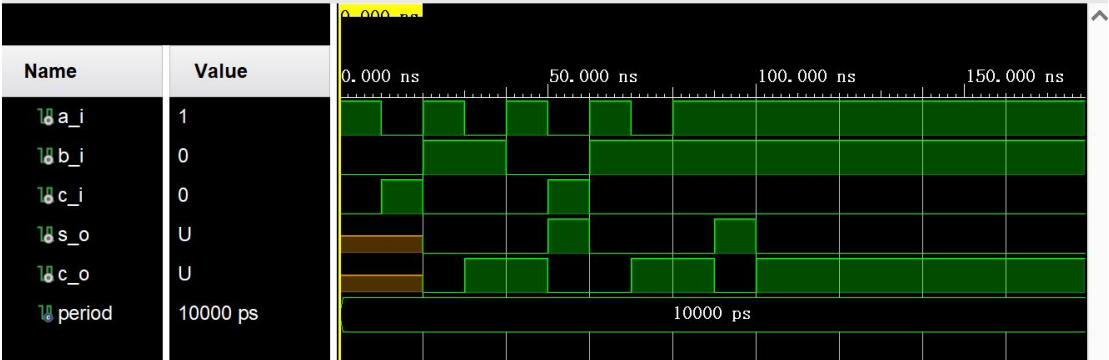
```

实验结果：

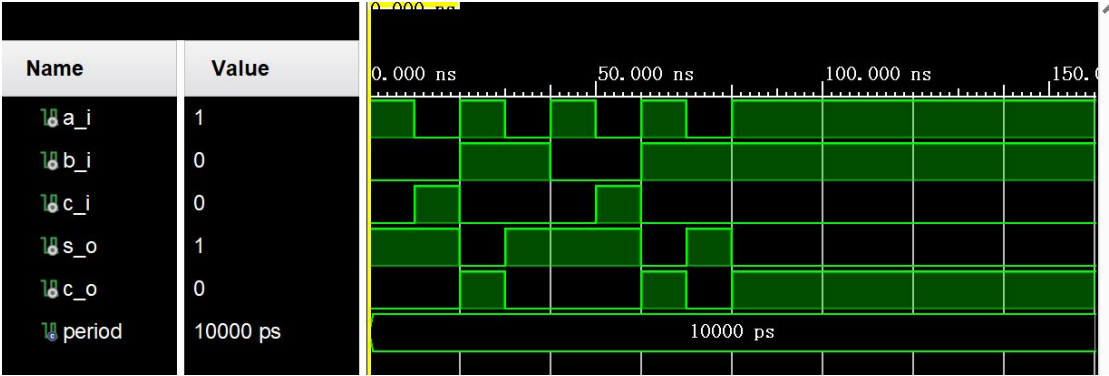
RTL analysis schematic



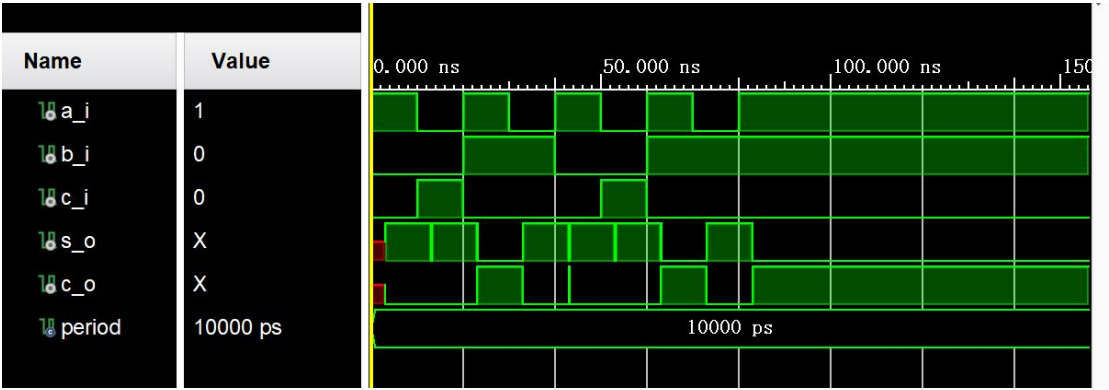
behavioral simulation



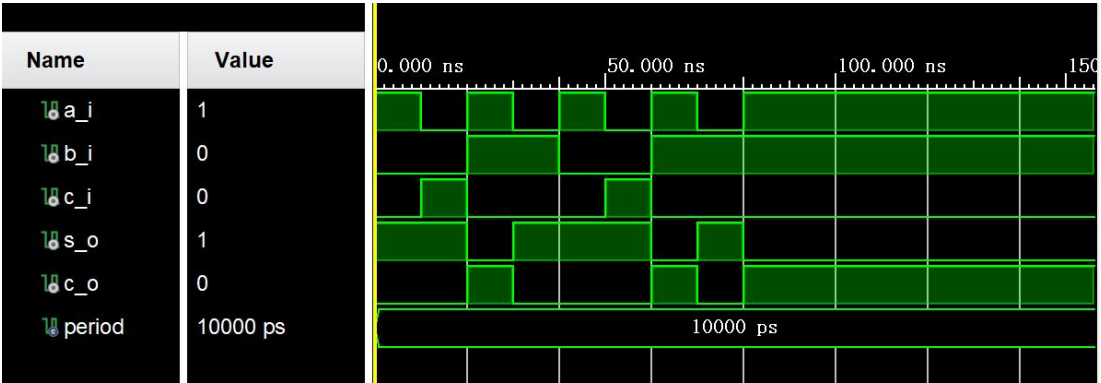
Post-synthesis simulation - functional



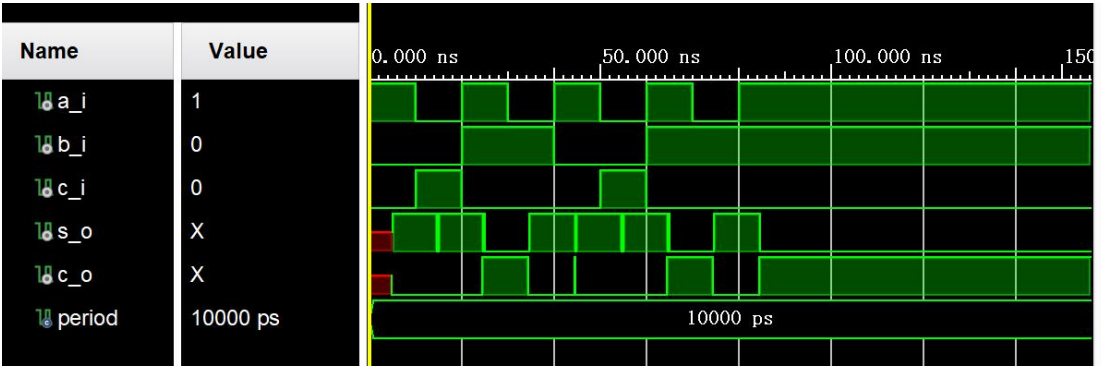
Post-synthesis simulation - timing



Post-implementation simulation - functional



Post-implementation simulation - timing



分析:

1. 仿真结果是否与上页图中的结果一致?

Behavioral simulation 的结果与图中结果一致。post-synthesis simulation 和 post-implementation simulation 的结果与图中结果不一致。

2. 加法计算结果是否和人们通常的认知一致?

不一致。

3. 如不一致，导致不一致的原因是什么?

A 和 B 为被加数，Cin 为来自低位的进位信号，Sum 为当前位的结果，Cout 为高位的进位信号。在通常的认知之中，当 A, B, Cin 的值发生变化时，Sum, Cout 的值应当没有延时地，在瞬间发生变化。



但是在 VHDL 代码之中，我们使用了如

```
s1 <= (A xor B) after gate_delay;
```

这样的并行信号赋值语句，对模块中的信号进行赋值，这些语句的特点是：它们的执行和赋值并不是同时进行的。在对 $A \text{ xor } B$ 进行计算之后，会在 `gate_delay` 的时间之后再计算结果赋给 `s1`。

根据这样的特点，我们就可以发现在 $0 \sim 10\text{ns}$ 时，`s1`, `s2`, `s3` 还并没有被赋值，因此它们的状态为 `uninitialized`，在 10ns 时，它们被赋值，它们被赋上的值是根据 0ns 时 `A`, `B`, `Cin` 的值来进行计算的。由此我们可以根据以下的赋值语句得到结论：`s1` 的值是 $(A \text{ xor } B)$ 的值延时 10ns 得到的，`s2` 的值是 $(Cin \text{ and } s1)$ 的值延时 10ns 得到的，`s3` 的值是 $(A \text{ and } B)$ 的值延时 10ns 得到的，`sum` 的值是 $(s1 \text{ xor } Cin)$ 的值延时 10ns 得到的，`cout` 的值是 $(s2 \text{ or } s3)$ 的值延时 10ns 得到的。

```
L1: s1 <= (A xor B) after gate_delay;
```

```
L2: s2 <= (Cin and s1) after gate_delay;
```

```
L3: s3 <= (A and B) after gate_delay;
```

```
L4: sum <= (s1 xor Cin) after gate_delay;
```

```
L5: cout <= (s2 or s3) after gate_delay;
```

根据以上的分析我们自然可以发现：计算结果和人们通常的认知并不一致，并且符合以上提到的规律。

4. 3 种仿真结果有什么不同？

Behavioral simulation 的仿真波形和通常认知的全加器的功能不同，但 post-synthesis simulation 以及 post-implementation simulation 的仿真波形和我们通常认知的全加器的功能相同。

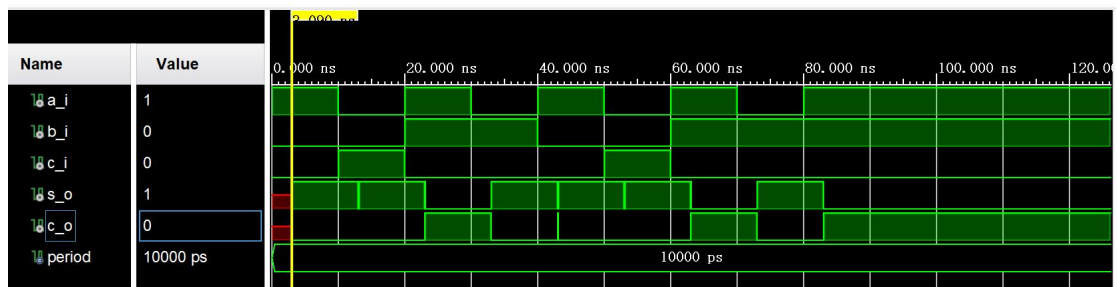


图 1 Post-synthesis simulation - timing

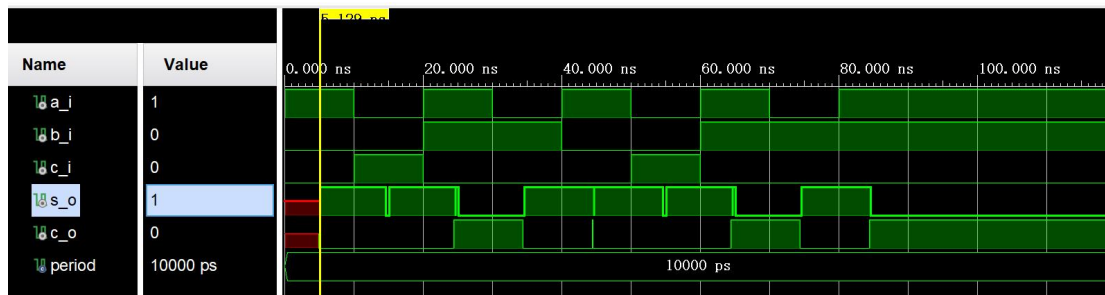


图 2 Post-implementation simulation - timing

同时,比较 post-synthesis simulation - timing 和 post-implementation simulation - timing 的仿真波形, 我们可以发现它们的延时并不相同, post-synthesis 中延时为 2.000ns, post-implementation 中延时为 5.120ns。

5. 为什么会有不同?

和在进行 behavioral simulation 时不同, 当在 VHDL 中进行 post-synthesis simulation 和 post-implementation simulation 时, 并行信号赋值语句的 after 项通常不会再产生延时。在综合和实现阶段, 综合器会将 VHDL 代码转换为硬件结构, 但不会考虑 after 项指定的延时, 因为硬件的实际延时是由物理特性(如门延迟、布线延迟等)决定的, 而不是由代码中的 after 项控制。

至于 synthesis 和 implementation 时得到的不同的时延, 这是因为 synthesis 将高层次设计转换为门级网表, 而 implementation 将门级网表映射到具体的 FPGA 器件上, 关注物理实现和布局布线。时延的不同实际上是门延迟和布线延迟时间的不同。

6. 你是否在结果看到了一些尖峰?

在 post-synthesis simulation - timing 和 post-implementation simulation - timing 的仿真波形中可以看到一些尖峰。

7. 这些尖峰是由什么引起的?

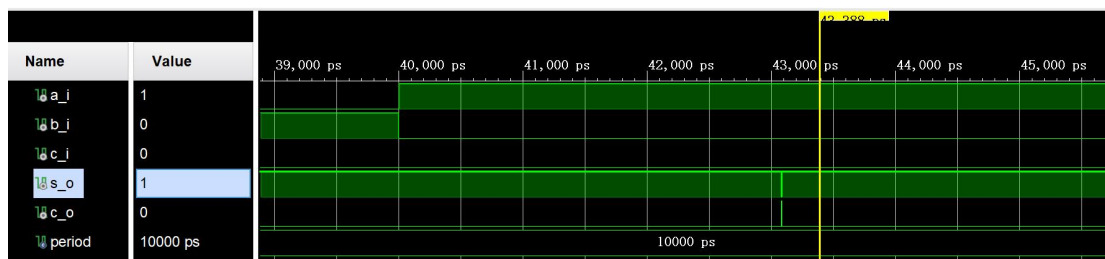


图 3 post-synthesis simulation - timing

尖峰是由电路中的竞争-冒险引起的, 观察 post-synthesis simulation - timing 的波形我们可以发现, 在 40,000 ps 时刻, 输入信号 A 由 0 跳变为 1, 输入信号 B 由 1 跳变为 0。根据数字电路的知识, 我们知道, 当两个输入信号同时向相反方向的逻辑电平跳变时, 电路中就会出现竞争-冒险现象, 产生尖峰脉冲。